

LSTM per Sentiment Analysis

Metodi di Ottimizzazione per Big Data
Anno Accademico 2024/2025

Federico Scordo
Matricola 0368288

Indice

1	Contesto, Obiettivi e Struttura del Progetto	2
2	Preprocessing e Gestione dei Dati	3
2.1	Pulizia del Testo, Tokenizzazione e Filtrazione Stop-words	3
2.2	Padding Dinamico e Partizionamento del Dataset	3
2.3	Costruzione del Vocabolario e Vettorizzazione	3
3	Architettura della Rete LSTM	5
3.1	Inizializzazione dei Parametri e Configurazione delle Matrici	5
3.2	Algoritmo di Forward Pass	5
3.3	Backward Propagation Through Time (BPTT)	6
4	Validazione e Ricerca degli Iperparametri (K-Fold Grid-Search)	8
4.1	Partizionamento dei Dati	8
4.2	Infrastruttura di training	8
5	Algoritmi di Ottimizzazione	10
5.1	SGD con Momentum di Nesterov	10
5.2	Adagrad (Adaptive Gradient)	10
5.3	RMSprop	11
5.4	AdamW (Adam with Weight Decay)	11
6	Risultati e Conclusioni	13
6.1	Conclusioni	14

1 Contesto, Obiettivi e Struttura del Progetto

Il presente progetto nasce con l'obiettivo di applicare gli algoritmi di ottimizzazione basati sulla discesa del gradiente, studiati durante il corso, ad un caso d'uso reale nell'ambito del *Machine Learning*, testandone l'efficacia nel minimizzare la funzione obiettivo di un modello complesso con un grande insieme di dati. A tale scopo si è deciso di affrontare un problema di **Sentiment Analysis**, ovvero la tecnica di *Natural Language Processing* (NLP) utilizzata per identificare se un corpo di testo è positivo o negativo. Essendo un processo ampiamente utilizzato per il monitoraggio dei social media, il dataset selezionato TwitterParsed è composto da una vasta raccolta di file testuali che rappresentano tweet effettuati dagli utenti dell'omonima piattaforma social.

Per elaborare efficacemente questa tipologia di dati, la scelta del modello di machine learning da utilizzare è ricaduta su una Rete Neurale Ricorrente (RNN) di tipo **Long Short-Term Memory (LSTM)**, implementata *from scratch*. Le LSTM sono progettate per mantenere il contesto informativo su sequenze lunghe di parole a differenza delle RNN tradizionali, che possono dimenticare informazioni precedenti in frasi con molte parole. Questo modello si basa su una cella di memoria interna coordinata da tre porte non lineari (Forget, Input e Output Gate) e si completa con un livello denso intermedio deputato all'estrazione di feature ad alto livello prima della classificazione binaria finale.

Siccome la scelta di una buona rete non è sufficiente a garantire la capacità di generalizzazione del modello, è stata adottata la tecnica della **K-Fold Cross-Validation** per identificare gli iperparametri ottimali di addestramento. Dividendo il dataset in K partizioni ed effettuando sessioni di addestramento incrociate è stato possibile stimare le performance del modello in modo indipendente dalla specifica suddivisione dei dati, potendo così individuare i valori degli iperparametri di *learning rate* e *regolarizzazione* adatti al modello.

Effettuata la configurazione teorica ideale tramite la validazione incrociata, il fulcro operativo del progetto si è focalizzato sulle strategie di addestramento della rete. In particolare si è scelto di confrontare quattro diversi algoritmi di ottimizzazione per la minimizzazione della funzione di perdita:

- **SGD con Momentum di Nesterov**: Rappresenta l'approccio classico all'ottimizzazione stocastica. Accelera la convergenza sfruttando l'inerzia dei gradienti passati per superare i minimi locali ed attenuare le oscillazioni nel cammino di discesa. Per garantire la stabilità, integra un tasso di apprendimento dinamico che decresce progressivamente nel tempo secondo le condizioni teoriche di convergenza.
- **Adagrad**: Algoritmo adattivo studiato per calibrare il passo di apprendimento in modo specifico per ogni singolo parametro, basandosi sulla storia dei suoi gradienti passati. Questo approccio si rivela ideale per dati sparsi come i testi, poiché assegna modifiche più decise ai parametri associati a parole rare. Soffre tuttavia di un rallentamento monotono e prematuro della discesa a causa dell'accumulo indefinito dei gradienti storici sulle lunghe distanze.
- **RMSprop**: Sviluppato esplicitamente per superare i limiti di Adagrad, sostituisce l'accumulo totale dei gradienti con una media mobile esponenziale. Grazie a questa intuizione, l'algoritmo tende a "dimenticare" la storia passata troppo remota, impedendo al denominatore di crescere oltre misura e consentendo al modello di mantenere un apprendimento efficace, fluido e stabile anche nelle fasi avanzate dell'addestramento.
- **AdamW**: Variante moderna che unisce i vantaggi dell'inerzia del Momentum alla calibrazione adattiva di RMSprop, introducendo una correzione analitica per i bias dovuti all'inizializzazione. La sua caratteristica chiave è il disaccoppiamento della regolarizzazione dal calcolo dei momenti: applicando la penalità direttamente sui pesi correnti anziché sul gradiente riscalato, evita interferenze numeriche distorsive, garantendo un addestramento uniforme tra le componenti strutturali e una superiore capacità di generalizzazione su dati non visti.

Quanto appena descritto mappa l'intero ciclo di sviluppo del software del progetto, anticipando a livello teorico ciò che sarà analizzato nelle sezioni successive del documento. Nei capitoli seguenti, l'intero workflow del progetto verrà analizzato sotto forma di codice MATLAB commentato: si partirà dal *Preprocessing e Gestione dei Dati* per la pulizia dei testi, si passerà alla struttura matematica nei file di *Architettura della Rete LSTM* interamente costruita da zero, per poi esaminare i driver di *Validazione e Ricerca degli Iperparametri* ed i singoli script degli *Algoritmi di Ottimizzazione*, concludendo con l'analisi dei *Risultati* empirici e delle curve di convergenza ottenute dai diversi ottimizzatori.

2 Preprocessing e Gestione dei Dati

Le reti neurali ricorrenti applicate al contesto del Sentiment Analysis richiedono una preventiva procedura di pulizia e preparazione del testo per mitigare il rumore intrinseco di tipi di testo come i tweet, caratterizzati da elementi non strutturati e linguaggio informale. La pipeline di preprocessing dei dati, interamente gestita dallo script `load_twitterdata.m`, trasforma i testi grezzi a lunghezza variabile in tensori numerici bidimensionali densi a dimensione fissa, ottimizzando al contempo l'addestramento tramite il calcolo matriciale parallelo in minibatch.

2.1 Pulizia del Testo, Tokenizzazione e Filtrazione Stop-words

Il dataset è organizzato nelle classi logiche 0 (sentiment negativo) e 1 (sentiment positivo) all'interno della directory `TwitterParsed`. Lo script applica in sequenza una serie di espressioni regolari (*regex*) per isolare ed estrarre unicamente i token alfabetici informativi utili alla classificazione.

Pipeline di pulizia tramite espressioni regolari e normalizzazione

```
% Estratto da load_twitterdata.m
txt = lower(txt);
txt = regexprep(txt, 'http\S+', '');      % Rimuove URL
txt = regexprep(txt, '@\S+', '');        % Rimuove Menzioni
% reduce repeated letters (e.g., loooove -> love)
txt = regexprep(txt, '([a-z])\1{2,}', '$1');
txt = regexprep(txt, '[^a-z_]', '');
txt = regexprep(txt, '[s+]', '_');
```

L'infrastruttura esegue la conversione del testo in minuscolo per uniformare i token, elimina URL, menzioni utente e normalizza le enfattizzazioni testuali social comprimendo le ripetizioni di tre o più lettere consecutive in una singola istanza. Infine, rimuove i caratteri non alfabetici e collassa gli spazi vuoti multipli. Il testo risultante viene diviso in singoli token verbali filtrando le *stop-words* (es. "the", "and", "by") tramite un dizionario statico, escludendo parole semanticamente povere che rallenterebbero la convergenza.

Tokenizzazione e rimozione delle stop-words semantiche

```
% Estratto da load_twitterdata.m
tokens = strsplit(strtrim(txt));
tokens = tokens(~cellfun('isempty', tokens));

% stop words removal
tokens = tokens(~ismember(tokens, stopWords));
```

2.2 Padding Dinamico e Partizionamento del Dataset

Per consentire il processamento in batch tramite strutture a dimensione costante, è necessario definire la lunghezza massima delle sequenze (`SEQ_LENGTH`). L'analisi delle lunghezze frequenti dei testi nel dataset, effettuata eseguendo lo script dedicato `analyze_seq_length.m`, ha portato alla scelta del **95° percentile**, minimizzando lo spreco computazionale causato dal *padding* per le frasi brevi, preservando il grosso dell'informazione.

Calcolo della lunghezza ottimale e split del dataset

```
% --- 2. SEQUENCE LENGTH ANALYSIS ---
p95 = prctile(docLengths, 95);
SEQ_LENGTH = ceil(p95) + 2;

% --- 3. DATASET SPLIT (70-15-15) ---
rng(42);
perm = randperm(numTotal);
idxSplit1 = floor(0.70 * numTotal);
idxSplit2 = floor(0.85 * numTotal);

docsTrain = allTokenizedDocs(perm(1:idxSplit1)); Y_Train = allLabels(perm(1:idxSplit1));
docsVal    = allTokenizedDocs(perm(idxSplit1+1:idxSplit2)); Y_Val    = allLabels(perm(idxSplit1+1:idxSplit2));
docsTest   = allTokenizedDocs(perm(idxSplit2+1:end)); Y_Test   = allLabels(perm(idxSplit2+1:end));
```

Subito dopo, l'integrità e riproducibilità del processo di addestramento vengono garantite tramite la configurazione del generatore di numeri casuali (`rng(42)`). Il dataset viene rimescolato in modo pseudocasuale (`randperm`) e partizionato rigidamente in tre set indipendenti: **Training Set (70%)** per la stima dei parametri, **Validation Set (15%)** per il monitoraggio della convergenza e **Test Set (15%)** per la verifica finale della capacità di generalizzazione.

2.3 Costruzione del Vocabolario e Vettorizzazione

Per convertire i token verbali in vettori numerici adatti ai layer di embedding, lo script estrae le parole più significative e costruisce un vocabolario indicizzato, che viene calcolato esclusivamente a partire dalle frasi appartenenti al set di addestramento (`docsTrain`):

Estrazione delle frequenze e mappatura del vocabolario

```
% --- 4. VOCABULARY CONSTRUCTION (Top 10000 words) ---
VOCAB_SIZE = 10000;
allTokens = [docsTrain{:}];
[uniqueWords, ~, ic] = unique(allTokens);
counts = histcounts(ic, 'BinMethod', 'integers');
[~, sortIdx] = sort(counts, 'descend');
sortedWords = uniqueWords(sortIdx);

topWords = sortedWords(1:min(VOCAB_SIZE, length(sortedWords)));
wordMap = containers.Map(topWords, 2:(length(topWords)+1));
```

Lo script calcola le occorrenze di ciascuna parola tramite le funzioni `unique` e `histcounts`, e ordina i termini in senso decrescente. Le **10.000 parole più frequenti** vengono inserite all'interno di una struttura `containers.Map`, una tabella hash che garantisce tempi di ricerca in tempo costante $O(1)$. La mappatura effettuata assegna gli indici numerici a partire dal valore `**2**` fino a `VOCAB_SIZE + 1`, in modo tale che l'indice 0 sia riservato al **Padding** e l'indice 1 al token speciale `<UNK>` (*Unknown word*).

La conversione effettiva delle sequenze di stringhe in tensori numerici avviene tramite la funzione interna `tokensToMatrix`, che viene applicata uniformemente sui tre set salvando il risultato nel file d'appoggio `ready_data.mat`:

Funzione di conversione e allocazione della matrice degli indici

```
% --- 5. VECTORIZATION ---
X_Train = tokensToMatrix(docsTrain, wordMap, SEQ_LENGTH);
X_Val   = tokensToMatrix(docsVal, wordMap, SEQ_LENGTH);
X_Test  = tokensToMatrix(docsTest, wordMap, SEQ_LENGTH);

% helper function for vectorization
function X = tokensToMatrix(docsCells, map, seqLen)
numDocs = length(docsCells);
X = zeros(numDocs, seqLen);
for i = 1:numDocs
    tokens = docsCells{i};
    L = min(length(tokens), seqLen);
    for t = 1:L
        word = tokens{t};
        if isKey(map, word)
            X(i,t) = map(word);
        else
            X(i,t) = 1; % <UNK>
        end
    end
end
```

La funzione inizializza una matrice di zeri di dimensioni pari al numero di documenti per la lunghezza massima della sequenza, preallocando implicitamente il valore di **Padding (0)** su ogni cella. Successivamente, un ciclo `for` esamina ogni parola fino al limite massimo consentito. Se il termine è presente nella tabella hash del vocabolario, la cella riceve il rispettivo indice; se la parola non è presente viene mappata sull'indice **1** (`<UNK>`). Il dataset così strutturato è matematicamente pronto per alimentare i layer successivi.

3 Architettura della Rete LSTM

L'implementazione manuale della rete *Long Short-Term Memory* (LSTM) elabora sequenze di vettori di embedding, producendo un output scalare per la classificazione binaria ed è strutturata in tre script: `init_lstm.m`, `forward_lstm.m` e `backward_lstm.m`.

3.1 Inizializzazione dei Parametri e Configurazione delle Matrici

Lo script `init_lstm.m` stabilisce le dimensioni geometriche dei tensori della rete. Al fine di evitare i fenomeni di *vanishing* o *exploding gradient*, tutti i parametri pesati sono inizializzati seguendo il metodo di **Xavier (Glorot)**, che mantiene costante la varianza delle attivazioni e dei gradienti scalando i valori casuali estratti in base alle dimensioni di input e di output del singolo strato.

Configurazione degli iperparametri in `init_lstm.m`

```
% hyper-parameters configuration
input_size = VOCAB_SIZE + 2; embed_size = 200; hidden_size = 128; dense_size = 128; output_size = 1;
```

La variabile `input_size` include le due unità aggiuntive destinate alla gestione di padding (0) e token sconosciuti (1). Le componenti logiche del sistema vengono allocate tramite matrici.

L'Embedding layer (`We`) crea lo spazio geometrico continuo in cui le parole vengono proiettate, associando a ciascuno degli indici discreti un vettore denso di dimensione 200.

Le quattro porte della cella LSTM (*forget*, *input*, *lo stato candidato* *cell state*, *output*) allocano le matrici per l'input corrente (`W*`), le matrici ricorrenti per lo stato nascosto precedente (`U*`) ed i vettori di bias (`b*`), inizializzando forzatamente ad 1 il bias del Forget Gate (`bf`) per evitare l'evaporazione iniziale dei contesti lunghi.

Infine, un Intermediate Dense Layer (`W_dense`, `b_dense`) introduce una trasformazione non lineare per espandere le capacità rappresentative prima della proiezione all'Output Layer (`Why`, `by`).

Inizializzazione Xavier Glorot e allocazione dei parametri strutturali

```
% weight initialization (Xavier/Glorot)
xavier = @(n_in, n_out) (rand(n_in, n_out) * 2 * (sqrt(6)/sqrt(n_in+n_out))) - (sqrt(6)/sqrt(n_in+n_out));

% --- A. EMBEDDING LAYER ---
lstm_net.We = xavier(input_size, embed_size);

% --- B. LSTM GATES for current input: Wf, Wi, Wc, Wo ---
lstm_net.Wf = xavier(embed_size, hidden_size); lstm_net.Uf = xavier(hidden_size, hidden_size); lstm_net.bf = ones(1, hidden_size);
lstm_net.W* = xavier(embed_size, hidden_size); lstm_net.U* = xavier(hidden_size, hidden_size); lstm_net.b* = zeros(1, hidden_size);

% --- C. INTERMEDIATE DENSE LAYER ---
lstm_net.W_dense = xavier(hidden_size, dense_size); lstm_net.b_dense = zeros(1, dense_size);

% --- D. OUTPUT LAYER ---
lstm_net.Why = xavier(dense_size, output_size); lstm_net.by = zeros(1, output_size);
```

3.2 Algoritmo di Forward Pass

Il passaggio in avanti è implementato nella funzione `forward_lstm.m`. Lo script riceve in input la matrice degli indici dei token `x_batch` di dimensioni $N \times T$ (dove N indica l'ampiezza del batch e T la lunghezza della sequenza) e restituisce le probabilità stimate insieme a una struttura `cache`, indispensabile per memorizzare le attivazioni intermedie che verranno riutilizzate durante la fase di retropropagazione dell'errore.

Nella prima parte della funzione viene allocato lo spazio in memoria tramite tensori tridimensionali per gli stati della cella e si implementa il ciclo di calcolo lungo la dimensione temporale:

Allocazione e ciclo di ricorrenza con gestione del padding in `forward_lstm.m`

```
% initialization
[N, T] = size(x_batch); H = size(net.Wf, 2); D = size(net.We, 2);

% hidden and cell states (3D tensor for batch processing)
h_states = zeros(N, H, T + 1); c_states = zeros(N, H, T + 1);

% gates_f, gates_i, gates_c, gates_o and dropout cache
gates_* = zeros(N, H, T); x_vectors = zeros(N, D, T); dropout_masks = zeros(N, H, T);

sig = @(x) 1 ./ (1 + exp(-x));

% recurrence loop
for t = 1:T
    idx = x_batch(:, t); mask_padding = (idx > 0);

    % embedding look-up with padding handling
    idx_safe = idx; idx_safe(idx == 0) = 1; xt(~mask_padding, :) = 0;

    h_prev = h_states(:, :, t); c_prev = c_states(:, :, t);
```

```

% gate computations: function sig for f, i, o and function tanh for c_tilde
f = sig(xt * net.Wf + h_prev * net.Uf + net.bf); i = ...; o = ...; c_tilde = ...;

% state updates
c_curr_raw = (f .* c_prev) + (i .* c_tilde); h_curr_raw = o .* tanh(c_curr_raw);

% dropout application
if dropout_rate > 0
m = (rand(N, H) > dropout_rate); scale = 1 / (1 - dropout_rate);
h_curr_raw = h_curr_raw .* m * scale; dropout_masks(:, :, t) = m * scale;
end

% padding logic: maintain previous state for padded tokens
h_states(:, :, t+1) = mask_padding .* h_curr_raw + (~mask_padding) .* h_prev;
c_states(:, :, t+1) = mask_padding .* c_curr_raw + (~mask_padding) .* c_prev;

% cache current step
gates_f(:, :, t) = f; gates_i(:, :, t) = i; gates_c(:, :, t) = c_tilde; gates_o(:, :, t) = o;
x_vectors(:, :, t) = xt;
end

```

Moltiplicando la matrice di input $\mathbf{x_t}$ ($N \times D$) e lo stato precedente $\mathbf{h_prev}$ ($N \times H$) direttamente per le matrici dei pesi della rete, le quattro porte logiche vengono stimate contemporaneamente per tutti gli N campioni del batch.

Poiché la matrice $\mathbf{W_e}$ dell'embedding non possiede una riga associata all'indice fittizio 0, l'algoritmo crea un vettore di indici sicuro (`idx_safe`) sostituendo gli zeri con degli uni. Successivamente, applicando l'operatore logico negato `~mask_padding`, azzerà le righe corrispondenti ai token di riempimento. Nello step finale dell'aggiornamento, se il token corrente è di padding, il mascheramento bypassa i valori "raw" appena calcolati, copiando per trasporto statico gli stati del passo precedente (`h_prev` e `c_prev`). Ciò congela l'evoluzione della cella e previene la corruzione delle informazioni ad opera degli zeri.

Se il tasso di rimozione è positivo, viene generata una maschera booleana `m`. I neuroni attivi vengono scalati per un fattore di compensazione pari a $1/(1-\text{dropout_rate})$, per stabilizzare il valore atteso delle attivazioni evitando di dover riscaldare i pesi in fase di test.

Al termine del ciclo temporale, l'ultimo stato nascosto `h_final` condensa l'informazione semantica accumulata e viene proiettato verso lo strato di classificazione binaria.

Calcolo dei logits, attivazione finale e caching in forward_lstm.m

```

% final output layer
h_final = h_states(:, :, end);

% check for optional dense layer
if isfield(net, 'W_dense')
h_dense = tanh(h_final * net.W_dense + net.b_dense); logits = (h_dense * net.Why) + net.by;
else
logits = (h_final * net.Why) + net.by;
end

% final activation
prob = sig(logits);

% pack cache structure
cache.x_batch = x_batch; ...
if isfield(net, 'W_dense'), cache.h_dense = h_dense; end

```

L'algoritmo verifica dinamicamente la presenza del campo `W_dense` all'interno della struttura della rete e, se presente, lo stato nascosto finale `h_final` subisce una compressione lineare seguita da una mappatura non lineare tramite la funzione tangente iperbolica (`tanh`), generando il vettore `h_dense`.

I coefficienti estratti vengono infine ridotti ad un unico vettore di combinazioni lineari (`logits`), che la funzione sigmoide trasforma in una distribuzione di probabilità schiacciata nell'intervallo aperto $[0, 1]$ per estrarre la probabilità attesa del sentiment. Tutti i tensori calcolati nel tempo vengono così impacchettati nella struttura `cache` per essere passati alla funzione di backpropagation.

3.3 Backward Propagation Through Time (BPTT)

La funzione `backward_lstm.m` esegue il calcolo analitico inverso dei gradienti tramite l'algoritmo Backpropagation Through Time (BPTT). Lo script riceve in input la matrice dei dati `x_batch`, le etichette reali `target` ($N \times 1$), i parametri attuali della rete e la struttura `cache`. La prima macro-sezione calcola la derivata della funzione di perdita (*Binary Cross-Entropy*) proiettando l'errore a ritroso attraverso il livello denso.

Calcolo dell'errore di output in backward_lstm.m

```

% extraction of dimensions and cache
% ...
% output error (N x 1)
dy = (cache.prob - target);

% --- 1. DENSE LAYER / OUTPUT GRADIENTS ---
if isfield(net, 'W_dense')
h_dense = cache.h_dense; grads.dWhy = (h_dense' * dy); grads.dby = sum(dy, 1); dh_dense = dy * net.Why';

```

```

% backpropagation through tanh of dense layer
d_act = dh_dense .* (1 - h_dense.^2);

grads.dW_dense = (h_s(:, :, end)' * d_act); grads.db_dense = sum(d_act, 1); dh_next = d_act * net.W_dense';
else
h_final = h_s(:, :, end); grads.dWhy = (h_final' * dy); grads.dby = sum(dy, 1); dh_next = dy * net.Why';
end

% --- 2. LSTM PARAMETERS GRADIENTS INITIALIZATION ---
% ...

```

Dopo aver preallocato le matrici dei gradienti globali per ciascun parametro della rete, il ciclo invertito si muove a ritroso lungo l'asse temporale calcolando i gradienti locali delle quattro porte e accumulandoli progressivamente nelle rispettive matrici. Infine viene normalizzato ogni gradiente calcolato per la dimensione del batch:

Ciclo di BPTT e normalizzazione in backward_lstm.m

```

% --- 3. BACKPROPAGATION THROUGH TIME (BPTT) ---
for t = T:-1:1
mask_p = (x_batch(:, t) > 0);

% padding logic: skip gradients for padded tokens
% dh_pass/dc_pass pass the gradient through without modification for padded tokens
dh_pass = dh_next .* (~mask_p); dc_pass = dc_next .* (~mask_p);

% filter dh_next for real tokens
dh_current = dh_next .* mask_p;

if use_dropout
dh_current = dh_current .* dropout_masks(:, :, t);
end

h_prev = h_s(:, :, t); c_curr = c_s(:, :, t+1); c_prev = c_s(:, :, t);
f = f_gate(:, :, t); i = i_gate(:, :, t); g = c_gate(:, :, t); o = o_gate(:, :, t);
xt = x_vecs(:, :, t);

tanh_c = tanh(c_curr);

% gates gradients
do_raw = (dh_current .* tanh_c) .* (o .* (1 - o));
dc_curr = (dh_current .* o .* (1 - tanh_c.^2)) + dc_next .* mask_p;

dg_raw = (dc_curr .* i) .* (1 - g.^2); di_raw = (dc_curr .* g) .* (i .* (1 - i));
df_raw = (dc_curr .* c_prev) .* (f .* (1 - f));

% parameter accumulation for grads.dWo, grads.dWc, grads.dWi and grads.dWf
grads.dW* = grads.dW* + xt' * d*_raw; grads.dU* = grads.dU* + h_prev' * d*_raw; grads.db* = grads.db* + sum(
    d*_raw, 1);

dxt = df_raw * net.Wf' + di_raw * net.Wi' + dg_raw * net.Wc' + do_raw * net.Wo';

% update embedding gradients
for n = 1:N
if mask_p(n)
idx = x_batch(n, t); grads.dWe(idx, :) = grads.dWe(idx, :) + dxt(n, :);
end
end

% recurrence for t-1
dh_next = (df_raw * net.Uf' + di_raw * net.Ui' + dg_raw * net.Uc' + do_raw * net.Uo') + dh_pass;
dc_next = (dc_curr .* f) + dc_pass;
end

% --- 4. BATCH NORMALIZATION (Final average) ---
fn = fieldnames(grads);
% ...

```

Dall'analisi del codice si osserva che le variabili `do_raw`, `dg_raw`, `di_raw` e `df_raw` applicano la derivata della catena (*Chain Rule*) incorporando le derivate delle funzioni di attivazione: per le funzioni sigmoidi (porte f, i, o) viene applicata la relazione $y \cdot (1 - y)$, mentre per la tangente iperbolica (porta del candidato g) $1 - y^2$.

La gestione del padding agisce specularmente rispetto al forward pass. Se il token analizzato al passo t è un riempimento fittizio ($\sim \text{mask}_p$), l'errore accumulato non deve alterare i gradienti dei pesi. Per fare questo, i tensori `dh_pass` e `dc_pass` catturano il gradiente in arrivo e lo trasportano intatto al passo $t - 1$ tramite l'operatore $\sim \text{mask}_p$. Al contempo, la componente `dh_current` viene azzerata, congelando l'accumulo delle matrici `grads`.

Il gradiente rispetto all'input della cella (`dxt`) viene mappato a ritroso sulla matrice di look-up dell'embedding `dWe`. Poiché ogni riga di `We` corrisponde ad una parola specifica, un ciclo scorre le sole frasi reali del batch ($\text{mask}_p(n) = \text{true}$), individua l'indice numerico del token e accumula l'errore sulla riga corretta della matrice globale.

Nell'ultimo blocco, un ciclo dinamico estrae i nomi dei campi della struttura tramite la funzione `fieldnames` e divide ogni matrice accumulata per la dimensione del batch N . Questa operazione trasforma le somme dei gradienti in gradienti medi, disaccoppiando l'ampiezza dello step di ottimizzazione dalla dimensione del batch scelta per l'addestramento.

4 Validazione e Ricerca degli Iperparametri (K-Fold Grid-Search)

Nello script orchestratore `kfold_gridsearch.m`, viene implementata una procedura di *Grid Search* accoppiata alla validazione incrociata *K-Fold Cross-Validation*. L'obiettivo è individuare la combinazione ottimale tra la velocità di apprendimento (*Learning Rate*, α) ed il coefficiente di regolarizzazione (*Weight Decay*, λ), garantendo una valutazione che risulti imparziale ed indipendente da una specifica suddivisione dei dati.

4.1 Partizionamento dei Dati

Per massimizzare il volume di informazioni utilizzabili si fondono verticalmente i vettori del set di addestramento e di validazione originali tramite il comando `X_CV = [X_Train; X_Val]`. Il dataset unificato viene rimescolato in modo pseudocasuale sfruttando una permutazione degli indici generata da `randperm` e poi suddiviso in $K = 3$ partizioni simmetriche (*folds*) di ampiezza costante, definita dalla variabile `fold_size`.

Ciclicamente, ciascun fold viene isolato per ricoprire il ruolo di set di validazione, mentre le rimanenti $K - 1$ partizioni vengono concatenate per costituire il set di addestramento. L'accuratezza finale associata ad una determinata coppia di iperparametri viene calcolata unicamente al termine del ciclo come media aritmetica delle accuratèzze registrate nelle K iterazioni indipendenti.

Logica di rotational split per la Cross-Validation in `kfold_gridsearch.m`

```
% Suddivisione rotazionale per il fold k
val_start = (k-1)*fold_size + 1; val_end = k*fold_size;

idx_v = indices(val_start:val_end); % Indici per validazione
idx_t = indices;
idx_t(val_start:val_end) = []; % Indici per addestramento (rimanenti)

X_T_Fold = X_CV(idx_t, :); Y_T_Fold = Y_CV(idx_t); X_V_Fold = X_CV(idx_v, :); Y_V_Fold = Y_CV(idx_v);
```

La validazione incrociata prende in considerazione le combinazioni tra i parametri di un vettore di tre valori di learning rate (`LR.VALUES`) e quattro valori di penalizzazione (`LAMBDA.VALUES`). Poiché la valutazione di una singola configurazione richiede l'esecuzione di K cicli di addestramento della rete LSTM, il costo computazionale della Grid Search cresce in modo combinatorio, così per diminuire i tempi di calcolo lo script sfrutta il **Parallel Computing Toolbox** di MATLAB distribuendo il carico di lavoro sui diversi core della CPU. Tramite la funzione `meshgrid`, le griglie discrete dei parametri vengono generate in uno spazio bidimensionale e successivamente linearizzate tramite l'operatore di srotolamento (`:`) in una matrice `combinations` di dimensioni 12×2 . Ciascuna riga rappresenta un'istanza computazionale isolata e autosufficiente, che viene assegnata dinamicamente a un *worker* del pool parallelo.

Parallelizzazione della Grid Search tramite `parfor` in `kfold_gridsearch.m`

```
% Loop parallelo sulle combinazioni di iperparametri
parfor i = 1:num_comb
    cur_lr = combinations(i, 1); cur_lambda = combinations(i, 2); local_fold_accs = zeros(K_FOLDS, 1);

    for k = 1:K_FOLDS
        % ... (Logica di split rotazionale) ...
        % Addestramento del singolo fold tramite la funzione di training relativa all'ottimizzatore selezionato
        local_fold_accs(k) = train_fold_generic(X_T_Fold, Y_T_Fold, X_V_Fold, Y_V_Fold, cur_lambda, cur_lr,
            init_net_file, OPTIMIZER);
    end
    % Accuratezza media per la combinazione corrente
    results_acc(i) = mean(local_fold_accs);
end
```

L'allocazione della variabile `local_fold_accs` all'interno del loop parallelo la qualifica come variabile locale, garantendo che i thread operino in zone di memoria protette e prive di conflitti di scrittura. Al completamento di tutte le iterazioni del ciclo interno, il valore medio viene archiviato nel vettore globale `results_acc`, ed i coefficienti ottimali vengono estratti determinando l'indice del valore massimo assoluto.

4.2 Infrastruttura di training

Per garantire l'equità e la riproducibilità del confronto tra i diversi ottimizzatori, lo script orchestratore delega l'intero ciclo di addestramento locale delle partizioni alla funzione `train_fold_generic.m`. Questa funzione incapsula la logica del ciclo delle epoche e l'aggiornamento dei parametri, assicurando che ogni algoritmo selezionato operi con le stesse condizioni.

All'avvio di ogni epoca, i campioni di addestramento del fold corrente vengono rimescolati tramite una permutazione casuale (`randperm`) per alterare l'ordine di presentazione dei minibatch. Successivamente, per ogni passo iterativo, il codice esegue il forward pass per determinare le attivazioni intermedie e la BPTT per calcolare i gradienti grezzi.

Prima di applicare le regole di ottimizzazione, lo script implementa un **Global Gradient Clipping** che funge da barriera di sicurezza numerica per la stabilità delle reti. Il codice calcola la norma $L2$ globale (`gnorm`)

combinando i quadrati di tutti i parametri estratti dinamicamente tramite la funzione `fieldnames` e se questa supera la soglia unitaria ($\text{CLIP_NORM} = 1.0$), viene calcolato un coefficiente di scala che riduce proporzionalmente l'intensità del vettore dei gradienti. Questa operazione previene l'*exploding gradient*, impedendo che oscillazioni numeriche distruggano i pesi Xavier preallocati.

Struttura di `train_fold_generic.m`

```
function best_val_acc = train_fold_generic(X_T, Y_T, X_V, Y_V, lambda_val, lr_val, init_net_file,
    optimizer_type)
% load initial master network weights
loaded = load(init_net_file); net = loaded.lstm_net;

% --- BASE HYPER-PARAMETERS ---
BATCH_SIZE = 32; EPOCHS = 3; CLIP_NORM = 1.0; DROPOUT_RATE = 0.3; EPSILON = 1e-8;

% initialize optimizer-specific states
params = fieldnames(net);
state1 = struct(); % v for sgd/adamw, G for adagrad/rmsprop
state2 = struct(); % v_t for adamw
for k = 1:numel(params)
    state1.(params{k}) = zeros(size(net.(params{k}))); state2.(params{k}) = zeros(size(net.(params{k})));
end

num_train = size(X_T, 1); num_val = size(X_V, 1); val_acc_history = []; global_iter = 0;

% training loop
for epoch = 1:EPOCHS
    perm = randperm(num_train); X_shuffled = X_T(perm, :); Y_shuffled = Y_T(perm);

    for i = 1:BATCH_SIZE:num_train
        global_iter = global_iter + 1; idx_end = min(i + BATCH_SIZE - 1, num_train);
        x_batch = X_shuffled(i:idx_end, :); y_batch = Y_shuffled(i:idx_end);

        % forward pass with dropout
        [~, cache] = forward_lstm(x_batch, net, DROPOUT_RATE);
        % backward pass
        grads = backward_lstm(x_batch, y_batch, net, cache);

        % 1. Global Gradient Clipping
        gnorm_sq = 0;
        for k = 1:numel(params)
            gp = grads.([ 'd' params{k} ]); gnorm_sq = gnorm_sq + sum(gp(:).^2);
        end
        gnorm = sqrt(gnorm_sq); scale = min(1.0, CLIP_NORM / (gnorm + 1e-6));

        % 2. Update Logic
        for k = 1:numel(params)
            p = params{k}; g = grads.([ 'd' p ]) * scale;

            switch lower(optimizer_type: 'sgd', 'adagrad', 'rmsprop', 'adamw')
            case 'sgd'
                % (sgd update) ...
                % ... other case block
            end
            end
        end
        % end of epoch validation
        correct_val = 0;
        for v = 1:BATCH_SIZE:num_val
            v_end = min(v + BATCH_SIZE - 1, num_val); [p_v, ~] = forward_lstm(X_V(v:v_end, :), net, 0);
            correct_val = correct_val + sum(round(p_v) == Y_V(v:v_end));
        end
        val_acc_history(end+1) = (correct_val / num_val) * 100;
    end
end
```

Il costruito `switch` mappa e adatta le equazioni dell'ottimizzatore selezionato. Per tutti gli algoritmi adattivi, la variabile `state1` mantiene in memoria la traccia dei momenti passati (la somma geometrica dei quadrati in Adagrad, la media mobile esponenziale in RMSprop o la stima del primo momento in AdamW), mentre `state2` archivia il secondo momento non centrato per l'aggiornamento di AdamW.

Nel blocco `case` di ogni ottimizzatore il *Decoupled Weight Decay* viene implementato isolando i bias tramite il costruito logico `if ~contains(p, 'b')`, dove la penalità agisce sulle sole matrici dei pesi prima dello step adattivo, proteggendo la regolarizzazione $L2$ da distorsioni di scalatura adattiva dei gradienti.

A fine epoca un ciclo esegue il forward pass sul set di validazione isolato, disattivando il dropout. Si calcola poi l'accuratezza finale ed il valore massimo storico registrato nelle tre epoche viene restituito come metrica di performance del fold.

Le configurazioni che hanno massimizzato l'accuratezza media sui fold paralleli sono riassunte nella tabella seguente e impostate come iperparametri per l'addestramento globale.

Ottimizzatore	Learning Rate (α)	Regolarizzazione (λ)	Acc. Media (K-Fold)
SGD Momentum	0.01	1e-4	76.67%
Adagrad	0.01	1e-4	77.61%
RMSprop	0.001	1e-3	77.81%
AdamW	0.001	1e-3	77.54%

5 Algoritmi di Ottimizzazione

In questa sezione viene mostrata l'implementazione pratica dei quattro algoritmi di ottimizzazione esaminati. Ciascun metodo è stato sviluppato in uno script indipendente (`train_lstm_*.m`) per condurre l'addestramento globale della rete LSTM per 10 epoche sull'intero dataset, in cui l'obiettivo comune è la minimizzazione della funzione di perdita di tipo **Binary Cross-Entropy**.

Tutti gli script condividono la medesima architettura software per la gestione dei minibatch, il monitoraggio della convergenza e la stabilizzazione dei gradienti, in parte già mostrata nella sezione precedente, come il calcolo della norma Euclidea globale di tutti i gradienti estratti dalla struttura.

Global Gradient Clipping

```
% 1. Global Gradient Norm Calculation
gnorm_sq = 0;
for k = 1:numel(params)
    p = params{k}; gp = grads.(['d' p]); gnorm_sq = gnorm_sq + sum(gp(:).^2);
end
gnorm = sqrt(gnorm_sq);
scale = min(1.0, CLIP_NORM / (gnorm + 1e-6));
```

Per gli algoritmi basati su tassi dinamici estranei a scaling adattivi individuali, il passo α si riduce ad ogni iterazione globale in base a un fattore di decadimento impostato a 10^{-4} , assicurando che il modello compia passi ampi nelle prime fasi per poi stabilizzarsi senza oscillare in prossimità del minimo.

Scheduler del Learning Rate decrescente nel tempo

```
% A. LR SCHEDULER (Diminishing Stepsize - Robbins-Monro)
decay_rate = 1e-4; lr = INIT_LR / (1 + decay_rate * global_iter);
```

Al termine di ogni epoca, viene azzerato il Dropout per misurare l'accuratezza della rete sul validation set dove lo script verifica se l'accuratezza corrente supera il massimo storico registrato fino all'epoca precedente; in caso positivo, i parametri aggiornati vengono congelati in un file locale `.mat`, realizzando un meccanismo implicito di *Early Stopping*.

Logica di salvataggio selettivo del miglior modello storico

```
if epoch == 1 || val_acc > max(val_acc_history(1:end-1))
    save('best_lstm_model.mat', 'lstm_net', 'loss_history', 'val_acc_history', 'VOCAB_SIZE', 'SEQ_LENGTH');
end
```

5.1 SGD con Momentum di Nesterov

L'algoritmo Stochastic Gradient Descent integrato con il moto di Nesterov, nello script `train_lstm_sgd_momentum.m`, alloca un tensore `v_state` inizializzato a zero per memorizzare le velocità storiche dei vettori. Il momentum di Nesterov valuta il gradiente proiettando idealmente i parametri nel futuro, sommando l'intensità della velocità corrente prima di determinare la direzione della discesa.

Aggiornamento e Decoupled Weight Decay in `train_lstm_sgd_momentum.m`

```
% 2. Parameter Update with Nesterov Logic
for k = 1:numel(params)
    p = params{k}; g = grads.(['d' p]) * scale;

    % momentum velocity update
    v_state.(p) = MOMENTUM * v_state.(p) + g;

    % Nesterov: update using future velocity
    update_dir = g + MOMENTUM * v_state.(p);

    if contains(p, 'b')
        lstm_net.(p) = lstm_net.(p) - lr * update_dir;
    else
        % Decoupled Weight Decay
        lstm_net.(p) = lstm_net.(p) - lr * update_dir - (lr * WEIGHT_DECAY * lstm_net.(p));
    end
end
```

L'inerzia accumulata in `v_state` viene moltiplicata per il coefficiente iperparametrico (`MOMENTUM = 0.95`) e sommata al gradiente scalato `g` per generare la direzione finale `update_dir`. L'applicazione del *Weight Decay* disaccoppiato interviene esclusivamente sulle matrici dei pesi tramite l'operatore logico negato `contains(p, 'b')`, escludendo dalla penalizzazione $L2$ la dinamica di stabilizzazione dei bias.

5.2 Adagrad (Adaptive Gradient)

Adagrad introduce la calibrazione dinamica del tasso di apprendimento per ogni singolo parametro della rete. Lo script `train_lstm_adagrad.m` alloca la struttura di memoria `G_state`, inizializzata a zero, per archiviare la somma geometrica progressiva dei quadrati dei gradienti registrati dall'inizio dell'addestramento.

```

for k = 1:numel(params)
p = params{k}; g = grads(['d' p]) * scale;

% 1. accumulate squared gradients: G_t = G_t-1 + g^2
G_state.(p) = G_state.(p) + g.^2;

% 2. calculate adaptive learning rate: lr_t = eta / sqrt(G_t + epsilon)
adaptive_lr = LEARNING_RATE ./ (sqrt(G_state.(p)) + EPSILON);

% decoupled weight decay (applied before update)
if ~contains(p, 'b')
lstm_net.(p) = lstm_net.(p) * (1 - LEARNING_RATE * WEIGHT_DECAY);
end

% 3. parameter update: theta = theta - lr_t * g
lstm_net.(p) = lstm_net.(p) - adaptive_lr .* g;
end

```

L'elevamento al quadrato ($g.^2$) costringe la matrice `G_state.(p)` ad una crescita rigorosamente monotona. Il tasso di apprendimento globale (`LEARNING_RATE = 0.01`) viene diviso per la radice quadrata della matrice di accumulo, stabilizzata dal fattore `EPSILON = 1e-8` per impedire divisioni per zero. Prima dell'aggiornamento effettivo i pesi strutturali subiscono una contrazione lineare pari a $(1 - \text{LEARNING_RATE} * \text{WEIGHT_DECAY})$, salvaguardando la dinamica dall'interferenza numerica del denominatore adattivo.

5.3 RMSprop

RMSprop, nello script `train_lstm_rmsprop.m`, riutilizza la struttura `G_state` ma ne altera la dinamica di accumulo sostituendo la somma dei quadrati storici con una media mobile esponenziale smorzata dal fattore di decadimento memorizzato nella costante `GAMMA`.

Media mobile esponenziale dei quadrati e aggiornamento in train_lstm_rmsprop.m

```

for k = 1:numel(params)
p = params{k}; g = grads(['d' p]) * scale;

% 1. moving average of squared gradients: G_t = gamma*G_t-1 + (1-gamma)*g^2
G_state.(p) = GAMMA * G_state.(p) + (1 - GAMMA) * g.^2;

% 2. calculate adaptive learning rate: lr_t = eta / sqrt(G_t + epsilon)
adaptive_lr = LEARNING_RATE ./ (sqrt(G_state.(p)) + EPSILON);

% weight decay application
if ~contains(p, 'b')
lstm_net.(p) = lstm_net.(p) * (1 - LEARNING_RATE * WEIGHT_DECAY);
end

% 3. parameter update: theta = theta - lr_t * g
lstm_net.(p) = lstm_net.(p) - adaptive_lr .* g;
end

```

L'equazione (`G_state.(p)`) assegna un peso pari al 90% alla traccia dei quadrati passati ($\text{GAMMA} * \text{G_state}.(p)$) e un peso del 10% all'apporto del gradiente quadratico corrente ($(1 - \text{GAMMA}) * g.^2$). Questa combinazione lineare impedisce la crescita indefinita del denominatore. Di conseguenza, l'elaborazione di `adaptive_lr` consente di riscaldare il tasso globale (`LEARNING_RATE = 0.0005`) in modo flessibile e se il modello attraversa regioni a gradiente piatto, la media mobile decresce ed il passo si espande automaticamente, mantenendo l'addestramento reattivo.

5.4 AdamW (Adam with Weight Decay)

AdamW, nello script `train_lstm_adamW.m`, unifica le precedenti strategie allocando le strutture di memoria distinte `m_state` (primo momento) e `v_state` (secondo momento non centrato), entrambe inizializzate a zero per ciascun parametro.

Calcolo dei momenti, correzione dei bias e passo disaccoppiato in train_lstm_adamW.m

```

for k = 1:numel(params)
p = params{k}; g = grads(['d' p]) * scale;

% 1. first moment estimate (m_t)
m_state.(p) = beta1 * m_state.(p) + (1 - beta1) * g;

% 2. second moment estimate (v_t)
v_state.(p) = beta2 * v_state.(p) + (1 - beta2) * (g.^2);

% 3. bias correction
m_hat = m_state.(p) / (1 - beta1^global_iter); v_hat = v_state.(p) / (1 - beta2^global_iter);

% 4. update calculation
update_step = lr * (m_hat ./ (sqrt(v_hat) + epsilon));

% decoupled weight decay
if ~contains(p, 'b')
lstm_net.(p) = lstm_net.(p) - (lr * WEIGHT_DECAY * lstm_net.(p));
end

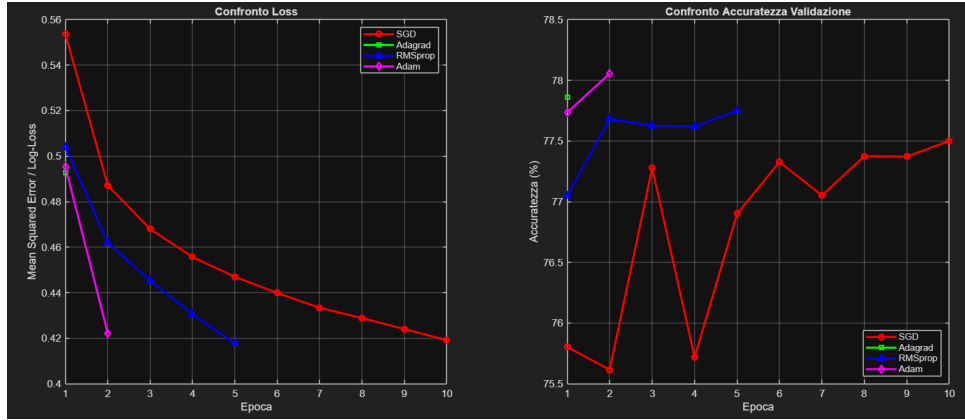
```

```
end
% 5. parameter update
lstm_net.(p) = lstm_net.(p) - update_step;
end
```

Le costanti `beta1 = 0.9` e `beta2 = 0.999` governano la memoria delle medie mobili. Poiché i tensori partono da zero, l'algoritmo compensa lo sbilanciamento iniziale verso l'origine calcolando `m_hat` e `v_hat`, ovvero dividendo le matrici correnti per i fattori di correzione $(1 - \beta_1^t)$ e $(1 - \beta_2^t)$, che convergono a 1 all'aumentare di `global_iter`. L'istruzione `update_step` definisce la direzione del passo basandosi sul rapporto elemento per elemento tra i momenti corretti. La penalizzazione *L2* viene applicata direttamente sulla matrice dei pesi correnti (`lstm_net.(p)`) in modo isolato dal calcolo di `update_step`, garantendo l'indipendenza tra la contrazione geometrica dei pesi e la riscalatura dei gradienti adattivi. Il passo globale `1r`, ricalcolato ad ogni ciclo, riduce progressivamente l'intensità sia della contrazione *L2* sia dello step adattivo, favorendo la stabilizzazione nelle fasi finali.

6 Risultati e Conclusioni

La valutazione del sistema è stata effettuata tramite gli script `confronto_finale.m` e `test_lstm.m`. Il primo genera un grafico che mette in relazione la dinamica di apprendimento (Loss e Accurazze di validazione) con le prestazioni effettive sul Test Set, permettendo di validare se la rapidità di convergenza di un ottimizzatore si traduce effettivamente in una maggiore capacità di generalizzazione.



Andamento comparativo della Loss e dell'accuratezza di validazione registrate nelle 10 epoche di training.

Il comportamento grafico evidenzia tre andamenti netti: i metodi adattivi (**AdamW** e **Adagrad**) mostrano una convergenza rapida raggiungendo il valore ottimale (78%) già alla seconda epoca grazie alla calibrazione sui vettori sparsi; **RMSprop** garantisce un'esplorazione regolare riducendo la loss fino al valore limite di 0.418 tramite la smorzatura della media esponenziale; lo **SGD con Momentum di Nesterov** manifesta oscillazioni marcate ed una traiettoria lenta, ma stabilizza il cammino nelle epoche finali grazie al decadimento di Robbins-Monro attestandosi al 77.5%.

Al fine di condurre un'analisi sul singolo modello, lo script `test_lstm.m` estrae la **Matrice di Confusione** e metriche di dettaglio come *Precision*, *Recall* e la loro media armonica *F1-Score* per garantire l'affidabilità della valutazione anche a fronte di classi potenzialmente sbilanciate nel Test Set.

Calcolo delle metriche in `test_lstm.m`

```
% --- 3. EVALUATION LOOP ---
for i = 1:BATCH_SIZE:num_test
    [prob, ~] = forward_lstm(x_batch, lstm_net, 0); pred = round(prob);

    % Calcolo vettoriale delle metriche nel batch
    correct = correct + sum(pred == y_batch);
    TP = TP + sum((pred == 1) & (y_batch == 1)); TN = TN + sum((pred == 0) & (y_batch == 0));
    FP = FP + sum((pred == 1) & (y_batch == 0)); FN = FN + sum((pred == 0) & (y_batch == 1));
end

% --- 4. METRICS CALCULATION ---
acc = (correct / num_test) * 100; prec = TP / (TP + FP + 1e-10);
rec = TP / (TP + FN + 1e-10); f1 = 2 * (prec * rec) / (prec + rec + 1e-10);
```

I dati estratti confermano la solidità del modello, che si attesta su livelli di accuratezza stabili in linea con i risultati di validazione, provando che la rete ha sviluppato un'effettiva capacità di generalizzazione e non ha semplicemente memorizzato il set di training.

6.1 Conclusioni

Il lavoro svolto ha dimostrato con successo la fattibilità e l'efficacia dell'implementazione *from scratch* di un'architettura complessa come la rete *Long Short-Term Memory* all'interno dell'ambiente MATLAB, superando la necessità di toolbox predefiniti per il deep learning. L'analisi sperimentale condotta evidenzia come il successo del classificatore sia il risultato di una sinergia tra una pipeline di preprocessing basata su espressioni regolari ed una scelta mirata degli iperparametri tramite la procedura di *K-Fold Cross-Validation*. I risultati raccolti confermano la netta superiorità dei metodi di ottimizzazione adattiva nel trattamento dei testi. Algoritmi come **AdamW** e **RMSprop** hanno mostrato una velocità di convergenza superiore rispetto allo Stochastic Gradient Descent tradizionale. Questo convalida l'efficacia teorica dello scaling del tasso di apprendimento basato sulla traccia passata del gradiente, cruciale per gestire l'elevata sparsità che caratterizza le matrici di embedding.

In conclusione, il progetto svolto non solo ha fornito una convalida di alcuni teoremi algoritmici visti durante il corso, ma ha anche evidenziato come tecniche di regolarizzazione come il *Decoupled Weight Decay* siano importanti nel preservare la capacità di generalizzazione del modello matematico.